

Cross-Machine QUIC Server-Side Migration

Cross-Machine QUIC Server-Side Migration

A research implementation proving that QUIC connections can be silently migrated between physically separate servers, demonstrating the feasibility of the QUIC-Exfil attack.

Built on Mozilla's [Necho](#) QUIC stack. Tested with real Firefox browser and four physical machines.

Table of Contents

- [1. Overview](#)
 - [2. Architecture](#)
 - [3. How It Works](#)
 - [4. Research Dimensions](#)
 - [4.1 When to Transfer \(Timing\)](#)
 - [4.2 How to Transfer \(Mechanism\)](#)
 - [5. Results](#)
 - [5.1 Combination Matrix](#)
 - [5.2 Multi-Metric Evaluation](#)
 - [5.3 Recommendations](#)
 - [6. Technical Details](#)
 - [6.1 Migration State Format](#)
 - [6.2 The Key Insight](#)
 - [6.3 Code Changes to Necho](#)
 - [7. Getting Started](#)
 - [7.1 Prerequisites](#)
 - [7.2 Build](#)
 - [7.3 Run](#)
 - [7.4 Switch Backends](#)
 - [7.5 Configuration Reference](#)
 - [7.6 Run Benchmarks](#)
 - [8. Project Structure](#)
 - [9. Documentation](#)
 - [10. Security Implications](#)
 - [11. References](#)
-

1. Overview

A client (Firefox or neqo-client) connects to a **primary server** via HTTP/3 over QUIC. During the TLS 1.3 handshake, the primary server advertises a **preferred address** ([RFC 9000 Section 9.6](#)) pointing to a completely different physical machine. After the handshake:

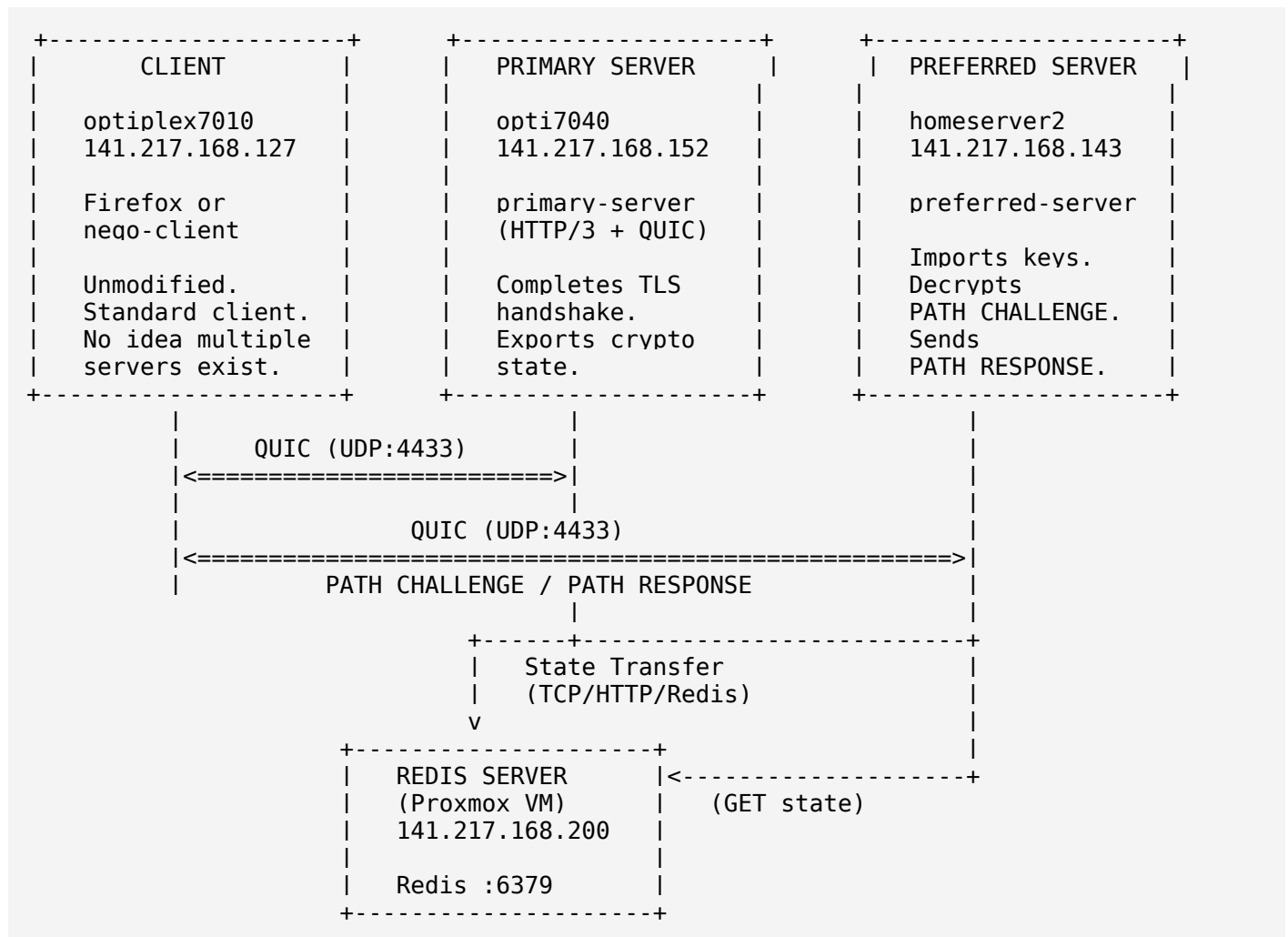
1. The primary server **exports** the TLS traffic secrets and connection IDs (~445 bytes)
2. It **transfers** them to the preferred server using one of four configurable mechanisms

3. The preferred server **imports** the keys and **decrypts** the client's PATH_CHALLENGE
4. It sends back a valid **PATH_RESPONSE**
5. The client's connection **silently migrates** to the preferred server

The client (and any firewall in between) has no indication that the connection moved to a different machine. This is the **QUIC-Exfil attack**.

2. Architecture

All experiments run across **four separate physical machines** on the same LAN:



Why four machines? To prove the migration is truly cross-machine, not within one process. The client is completely unmodified – it works with standard Firefox. The Redis server is on a dedicated Proxmox VM, keeping state transfer infrastructure independent from both QUIC servers.

[Back to top](#)

3. How It Works

Step-by-Step Migration Flow

```

Phase 1: QUIC Handshake
Client ---- ClientHello ----> Primary Server
Client <--- ServerHello ----- Primary Server
          (includes preferred address = 141.217.168.143:4433)
Client ---- Handshake Done -> Primary Server

Phase 2: State Transfer
Primary Server ---- 445 bytes (TLS secrets + CIDs) ----> Preferred Server
                  via TCP / HTTP / Redis

Phase 3: Path Validation
Client ---- PATH CHALLENGE (8 random bytes) -----> Preferred Server
Client <--- PATH_RESPONSE (same 8 bytes, encrypted) ---- Preferred Server

Phase 4: Migration Complete
Client now sends ALL traffic to Preferred Server.
Firewall sees normal encrypted QUIC packets.
No indication of migration.

```

[Back to top](#)

4. Research Dimensions

We investigate two independent dimensions of migration state transfer. Every combination is tested end-to-end.

4.1 When to Transfer (Timing)

Selected via `TRANSFER_TIMING` environment variable:

Timing	Env Value	Description
Immediate	<code>immediate</code>	Transfer state right after TLS handshake completes. Preferred server blocks and waits for state before listening for QUIC packets. Preferred server starts listening for QUIC packets immediately. State receiver runs in a background thread.
Lazy	<code>lazy</code>	Crypto import happens on-demand when the first unknown CID packet arrives.

Key difference: Immediate is simpler but always transfers state even if the client never migrates. Lazy only imports crypto when actually needed, but requires the state to persist (rules out some backends).

4.2 How to Transfer (Mechanism)

Selected via `STATE_TRANSFER` environment variable:

Mechanism	Env Value	Pattern	How it works
TCP Push	<code>tcp</code>	Direct push	Primary opens TCP socket to preferred, sends state bytes

Mechanism	Env Value	Pattern	How it works
HTTP Pull	http	On-demand pull	directly. One-way, one-time, connection closed after. Primary starts HTTP server exposing <code>/state</code> endpoint. Preferred sends HTTP GET to pull the state. Primary writes state to Redis with <code>SET</code> (CID-based key, configurable TTL). Preferred reads with <code>GET</code> . Delete after read.
Redis KV	redis_kv	Shared storage	Primary publishes state to Redis channel. Preferred subscribes and receives the message.
Redis Pub/Sub	redis_ps	Event-driven	

[Back to top](#)

5. Results

5.1 Combination Matrix

All 8 combinations of timing and mechanism tested end-to-end across four physical machines. **100% success rate.**

	TCP Push	HTTP Pull	Redis KV	Redis Pub/Sub
Immediate	PASS send: 734us	PASS send: 35ms	PASS send: 1.2ms	PASS send: 1.5ms
Lazy	PASS send: 479us import: 7us	PASS send: 25ms import: 24ms	PASS send: 1.5ms import: 4ms	PASS send: 1.5ms import: 425us

Note: Lazy timing works with TCP Push and Redis Pub/Sub because the receiver starts in a background thread at startup, ensuring it's ready when state arrives.

5.2 Multi-Metric Evaluation

Each combination scored on 8 research metrics (1-5 scale, 5 is best):

Combination	Lat	Sec	Scl	Rel	Cpl	Sim	Dep	Per	TOTAL
Imm + TCP Push	5	1	1	2	1	5	5	1	21
Imm + HTTP Pull	2	3	3	3	3	3	5	2	24
Imm + Redis KV	4	3	5	5	5	3	2	5	32
Imm + Redis PS	4	1	4	2	5	3	2	1	22
Lazy + TCP Push	5	2	1	3	1	3	5	1	21
Lazy + HTTP Pull	2	5	3	3	3	3	5	3	27
Lazy + Redis KV	4	3	5	5	5	3	2	5	32

	Lazy	+	Redis	PS		4		1		4		3		5		3		2		1		23	
+	-	-	-	-	+	-	-	-	-	-	+	-	-	-	+	-	-	-	+	-	-	-	+

What each metric measures:

Metric	Abbr	What it measures
Latency	Lat	How fast is the state transfer
Security	Sec	How well are TLS secrets protected during and after transfer
Scalability	Scl	Can it handle N primary servers and M preferred servers (N:M routing)
Reliability	Rel	Does it work if a server crashes, restarts, or the network hiccups
Coupling	Cpl	How independent are the servers from each other (loose = better)
Simplicity	Sim	How easy is it to implement and maintain
Dependencies	Dep	Does it require external infrastructure (Redis, etc.)
Persistence	Per	Does the state survive if a server restarts before migration completes

5.3 Recommendations

Use Case	Best Combination	Score	Why
Production deployment	Redis KV (either timing)	32/40	Highest in scalability, reliability, persistence. Supports N:M routing via CID-based keys. Works with both timing strategies. Secrets stay in primary server memory until explicitly pulled. Can add HTTPS. No third-party storage.
Security-critical	Lazy + HTTP Pull	27/40	Fastest latency (734us). Simplest code. No dependencies.
Demos and testing	Immediate + TCP Push	21/40	Same Redis dependency
Not recommended	Redis Pub/Sub	22-23	

Use Case

Best Combination

Score

Why

as KV but worse security (broadcasts secrets to all subscribers) and worse persistence.

[Back to top](#)

6. Technical Details

6.1 Migration State Format

The primary server exports approximately **445 bytes** after the TLS handshake:

Component	Size	Purpose
Magic header "MIGR"	4 bytes	Format identifier
QUIC version	4 bytes	Version 1 or Version 2
Write traffic secret	~34 bytes	Server-to-client encryption
Read traffic secret	~34 bytes	Client-to-server decryption
Next traffic secrets	~68 bytes	For future TLS key rotation
Cipher suite + metadata	~25 bytes	Algorithm IDs, packet number counters
8 local Connection IDs	~90 bytes	What the client uses to address us
Remote Connection IDs	~12 bytes	What we use to address the client
Client socket address	7 bytes	IP address + UDP port

6.2 The Key Insight

TLS 1.3 key derivation is **deterministic**. Given the same traffic secret, cipher suite, and HKDF labels, you always get the same AEAD encryption key and Header Protection key. This is guaranteed by [RFC 8446](#) and [RFC 5869](#).

```
// EXPORT (on primary server):
let secret_bytes = self.secret.key_data()?.to_vec(); // Raw TLS secret out

// IMPORT (on preferred server):
let secret = hkdf::import_key(TLS_VERSION_1_3, &secret_bytes)?; //
Reconstruct
let aead = Aead::new(TLS_VERSION_1_3, cipher, &secret, label)?; // Re-derive
let hpkey = hp::Key::extract(TLS_VERSION_1_3, cipher, &secret, &hplabel)?;
```

Both servers end up with **mathematically identical encryption keys**, even though the TLS handshake only happened on one of them.

6.3 Code Changes to Neqo

We modified 4 files and added 1 new module in Mozilla's `neqo-transport` crate:

File	Change	Why
<code>crypto.rs</code>	Added <code>secret: SymKey</code> field to <code>CryptoDxAppData</code>	Retain TLS traffic secret for export (normally discarded)
<code>migration_state.rs</code>	New module	Binary serialization format for all migration state
<code>connection/mod.rs</code>	Added <code>export_migration_state()</code>	Gather crypto + CIDs + client address into one struct
<code>cid.rs</code>	Added <code>sequence_number()</code> , <code>iter()</code> , <code>local_cids()</code>	Enumerate Connection IDs for export
<code>lib.rs</code>	Made <code>crypto</code> module public	Allow migration-test crate to access crypto types

Plus the `migration-test` crate: 2 server binaries + 4 pluggable transfer backends with a shared trait interface.

[Back to top](#)

7. Getting Started

7.1 Prerequisites

- **Rust toolchain** (rustup, cargo)
- **clang + libclang-dev** (for NSS compilation)
- **gyp, ninja-build, mercurial** (for NSS build)
- **Redis server** (for `redis_kv` and `redis_ps` backends only)
- **Four machines** on the same network (or adjust IPs): client, primary server, preferred server, Redis server

7.2 Build

```
# On most machines:
PKG_CONFIG_PATH=/nonexistent PKG_CONFIG_LIBDIR=/nonexistent cargo build --
release

# If libclang is in a non-standard path (e.g., homeserver2):
LIBCLANG_PATH=/usr/lib/llvm-14/lib PKG_CONFIG_PATH=/nonexistent
PKG_CONFIG_LIBDIR=/nonexistent cargo build --release
```

7.3 Run

Set the runtime library path first:

```
export LD_LIBRARY_PATH=$(find target/release/build -path '*/dist/Release/lib'
-type d | head -1)
```

Then start the servers in order:

```
# 1. Preferred server (homeserver2):
./target/release/preferred-server 141.217.168.143:4433 9999

# 2. Primary server (opti7040):
./target/release/primary-server 0.0.0.0:4433 141.217.168.143:4433
141.217.168.143:9999

# 3. Client (any machine):
./target/release/neqo-client https://141.217.168.152:4433/
```

For Firefox, see the [COMPREHENSIVE_GUIDE.pdf](#) for `about:config` settings and certificate import.

7.4 Switch Backends

```
# Example: Lazy timing + Redis KV
# Preferred server:
STATE_TRANSFER=redis kv REDIS_URL=141.217.168.200:6379 TRANSFER_TIMING=lazy \
./target/release/preferred-server 141.217.168.143:4433

# Primary server:
STATE_TRANSFER=redis kv REDIS_URL=141.217.168.200:6379 \
./target/release/primary-server 0.0.0.0:4433 141.217.168.143:4433
```

7.5 Configuration Reference

Variable	Values	Default	Description
STATE_TRANSFER	tcp, http, redis_kv, redis_ps	tcp	Transfer mechanism
TRANSFER_TIMING	immediate, lazy	immediate	When to import crypto
STATE_TCP_ADDR	host:port	-	TCP Push: where to send
STATE_TCP_PORT	port	9999	TCP Push: listen port
STATE_HTTP_PORT	port	9998	HTTP Pull: primary's HTTP port
STATE_HTTP_PRIMARY	host:port	-	HTTP Pull: where to pull from
REDIS_URL	host:port	141.217.168.200:6379	Redis server address (dedicated Proxmox VM)
REDIS_TTL	seconds	60	Redis KV: state expiry
INSTANCE_ID	string	default	Multi-server: CID routing key

7.6 Run Benchmarks


```
# Experiment 1: Single-pair latency (20 runs per backend)
./benchmarks/run_benchmark.sh 20

# Experiment 2: Multi-server (3 primary x 3 preferred)
./benchmarks/run_multiserver.sh

# Experiment 3: Concurrent clients stress test
./benchmarks/run_concurrent.sh 5
```

[Back to top](#)

8. Project Structure

```
quic-server-migration/
|
+-- README.md                # This file
+-- COMPREHENSIVE GUIDE.pdf   # Full 21-page technical guide
+-- COMPARISON_RESULTS.pdf    # 3-page comparison summary
|
+-- docs/
|   +-- PAPER SUMMARY.md      # QUIC-Exfil paper analysis
|   +-- *.svg                 # Architecture diagrams
|
+-- migration-test/          # OUR CODE (primary + preferred servers)
|   +-- src/
|       |   +-- primary server.rs    # HTTP/3 primary server
|       |   +-- preferred server.rs  # Preferred server (immediate + lazy)
|       |   +-- transfer/            # Pluggable state transfer backends
|       |       +-- mod.rs           # StateSender / StateReceiver traits
|       |       +-- tcp push.rs      # Backend 1: Direct TCP
|       |       +-- http pull.rs     # Backend 2: HTTP REST API
|       |       +-- redis kv.rs      # Backend 3: Redis SET/GET
|       |       +-- redis pubsub.rs  # Backend 4: Redis Pub/Sub
|       |       +-- file_backend.rs  # Backend 5: Shared file (not used)
|   +-- Cargo.toml
|
+-- benchmarks/              # Benchmark scripts
|   +-- run benchmark.sh      # Experiment 1: single-pair latency
|   +-- run multiserver.sh    # Experiment 2: N:M routing
|   +-- run concurrent.sh     # Experiment 3: stress test
|   +-- results/              # CSV output
|
+-- nego-transport/src/      # MODIFIED NEGO FILES
|   +-- crypto.rs            # TLS secret export/import
|   +-- migration state.rs   # State serialization (NEW)
|   +-- connection/mod.rs    # export migration_state()
|   +-- cid.rs               # CID accessors
|   +-- lib.rs               # Public crypto module
|
+-- nego-bin/                # Nego client/server binaries
+-- nego-http3/              # HTTP/3 implementation
+-- nego-common/             # Shared utilities
+-- nego-qpac/               # QPACK header compression
+-- nego-udp/                # UDP socket handling
+-- test-fixture/            # Test certificates and helpers
```

[Back to top](#)

9. Documentation

Document	Pages	Content
COMPREHENSIVE_GUIDE.pdf	21	Full guide: QUIC background, architecture, code changes, 4 timing strategies, 4 transfer mechanisms, When x How matrix, benchmark results, security analysis, multi-metric scoring, recommendations
COMPARISON_RESULTS.pdf	3	Focused summary: all 8 combinations, scoring table, recommendations
docs/PAPER_SUMMARY.md	-	Analysis of the original QUIC-Exfil attack research paper

10. Security Implications

This implementation demonstrates the **QUIC-Exfil attack**:

1. A malicious server accepts a QUIC connection from a victim client
2. It advertises a `preferred_address` pointing to an **attacker-controlled machine**
3. The TLS crypto state is transferred to the attacker's machine
4. The client **automatically migrates** – the attacker receives all traffic
5. The firewall sees only normal encrypted QUIC packets

Why firewalls cannot detect this:

- The `preferred_address` transport parameter is inside the **encrypted** TLS handshake
- `PATH_CHALLENGE` and `PATH_RESPONSE` are **encrypted** QUIC frames
- Post-migration traffic is **indistinguishable** from normal QUIC traffic
- ML-based classifiers achieve only **0-47% F1-score** against this attack

[Back to top](#)

11. References

- [RFC 9000 - QUIC Transport Protocol](#) – Section 9.6: Server's Preferred Address
 - [RFC 8446 - TLS 1.3](#) – Cryptographic handshake used by QUIC
 - [Mozilla Neqo](#) – Base QUIC implementation (Rust)
 - [QUIC-Exfil](#) – Related attack implementation by paper authors
-

License

Research project. Neqo modifications are based on Mozilla's Neqo ([Apache 2.0](#) / [MIT](#) dual license).